МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»
Центр образовательных программ топ-уровня в сфере
информационных технологий Образовательного центра Института №8

Отчет по лабораторной работе

по дисциплине «Операционные системы»

на тему: **«Обеспечение синхронизации между потоками»**

Выполнил:

студент гр. М8О-205БВ-25

Батурин Эдуард Дмитриевич

Проверил: Филимонов Н. С.

Оценка:

Дата:

Москва 2026

Цель работы: приобретение практических навыков в управление потоками в ОС и обеспечение синхронизации между потоками

Задание: Составить программу на языке Си, обрабатывающую данные в многопоточном режиме. При обработки использовать стандартные средства создания потоков операционной системы (Windows/Unix). Ограничение максимального количества потоков, работающих в один момент времени, должно быть задано ключом запуска вашей программы.

Ход работы

Я написал программу на си. Сначала был реализован простой алгоритм генерации псевдослучайных чисел.

```
static uint64_t splitmix64_next(uint64_t *state) {
    uint64_t value = (*state += UINT64_C(0x9e3779b97f4a7c15));

    value = (value ^ (value >> 30)) * UINT64_C(0xbf58476d1ce4e5b9);
    value = (value ^ (value >> 27)) * UINT64_C(0x94d049bb133111eb);
    return value ^ (value >> 31);
}

static double random_unit_interval(uint64_t *state) {
    return (double)(splitmix64_next(state) >> 11) * (1.0 / 9007199254740992.0);
}
```

Также были реализованы различные методы для парсинга ввода, а также алгоритм Монте-Карло.

```
typedef struct {
    uint64_t samples;
    uint64_t hits;
```

```
    uint64_t worker_index;
    uint64_t random_state;
} WorkerArguments;

static void *monte_carlo_worker(void *raw_arguments) {
    WorkerArguments *arguments = raw_arguments;
    uint64_t hits = 0;

    for (uint64_t sample = 0; sample < arguments->samples; sample++) {
        const double x = 2.0 * random_unit_interval(&arguments->random_state) -
1.0;
        const double y = 2.0 * random_unit_interval(&arguments->random_state) -
1.0;

        if (x * x + y * y <= 1.0) {
            hits++;
        }
    }

    arguments->hits = hits;
    return NULL;
}
```

В основном блоке программы потоки создаются с помощью pthread_create(&threads[index], NULL, monte_carlo_worker, &arguments[index]), а затем ожидаются с помощью pthread_join(threads[index], NULL).

С помощью hyperfine я замерил скорость исполнения программы в зависимости от количества доступных потоков.

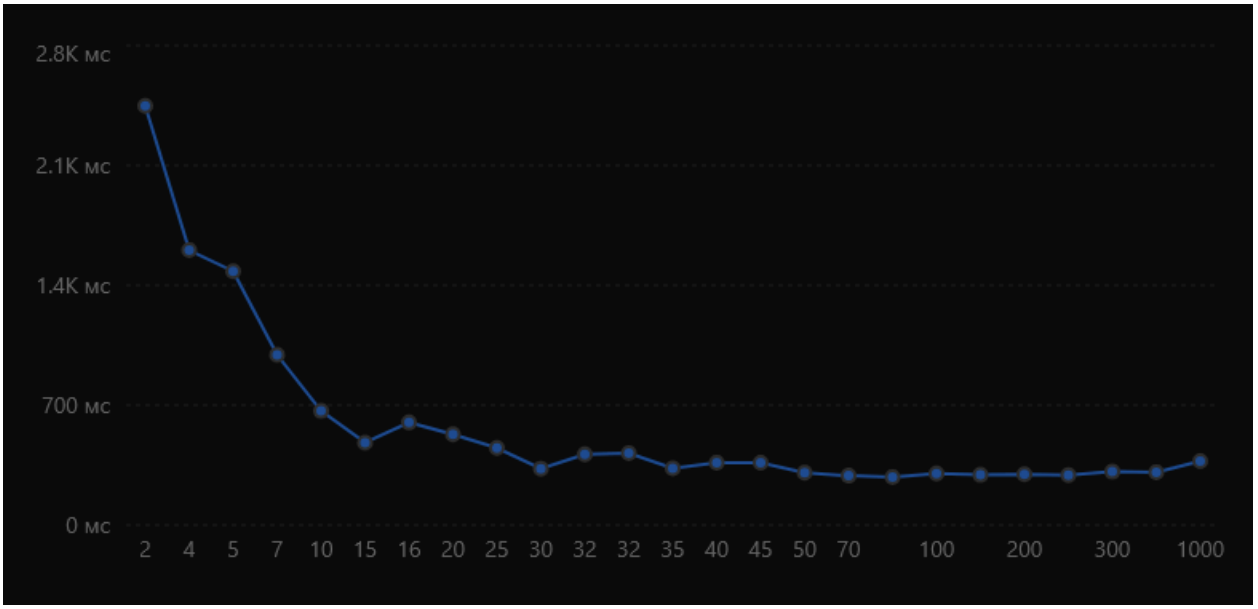


График 1. Зависимость времени исполнения от количества потоков на 100000000 элементах.

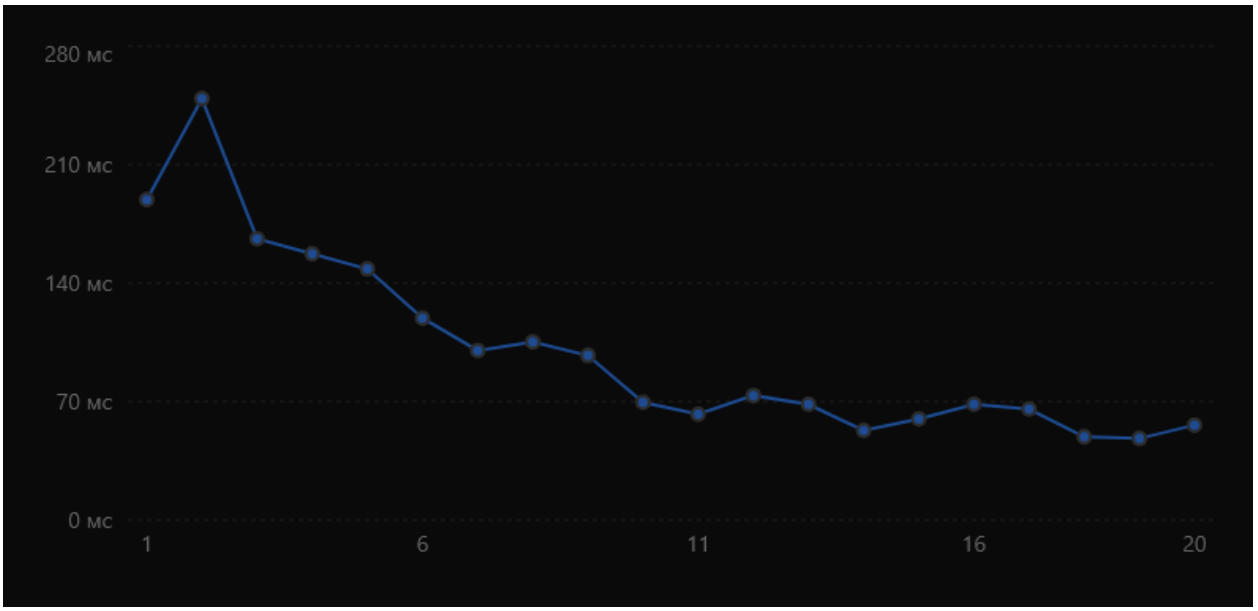


График 2. Зависимость времени исполнения от количества потоков на 10000000 элементах.

Вывод: я приобрел практические навыки в управлении потоками в ОС и обеспечении синхронизации между потоками.